Agentic Debugging: A Deep Research Literature Review

1. Executive Summary

This report reviews the research foundations for building an **agentic debugging system** — one that dynamically observes running programs, controls debuggers (PDB/GDB/LLDB), inspects runtime state, localizes faults, reasons about root cause, generates patches, and iteratively validates them. The central finding is that the field has bifurcated into two largely separate lineages: (1) a mature **static/repository repair** lineage (SWE-bench-driven agents such as SWE-agent, OpenHands, AutoCodeRover, Agentless) that reads code, error text, and test output but rarely touches a live debugger; and (2) a much smaller, newer **interactive debugger-control** lineage (ChatDBG, LDB, and 2025–2026 systems debug-gym, EnIGMA, FramePilot/ADI, Debug2Fix, SWE-Doctor) in which an LLM actually drives a runtime debugger.

Key conclusions: (a) Runtime/debugger information is demonstrably additive but not universally beneficial — Microsoft's debug-gym shows pdb access helps strong models yet can *harm* weaker ones. (b) Fault localization ranks suspicious locations; it is NOT the same as root-cause analysis, which requires causal/counterfactual reasoning (the Zeller delta-debugging lineage). (c) Passing tests ≠ correct patch — patch overfitting and SWE-bench contamination/leakage are well-documented. (d) A simple pipeline (Agentless) matches or beats complex multi-agent systems at far lower cost, so multi-agent superiority must not be assumed. (e) For a first prototype, Python + PDB is the best-evidenced choice, with deterministic tools for localization/trace capture and the LLM confined to hypothesis formation and patch proposal under a verifier.

This is a planning-phase review; recommendations distinguish literature-supported conclusions from engineering judgment and open experimental questions.

Coverage Map

Topic	Sources found	Full-text	Partial/abstract	Foundational coverage	Recent coverage	Confidence coverage sufficient
Debugging foundations	3	1	2	Moderate	Low	Medium
Automated debugging / delta debugging	3	2	1	Strong (Zeller)	Low	High
Fault localization (SBFL etc.)	5	2	3	Strong	Moderate	High

Topic	Sources found	Full-text	Partial/abstract	Foundational coverage	Recent coverage	Confidence coverage sufficient
Root-cause analysis	3	2	1	Strong (Zeller)	Low	Medium
Automated program repair (classic)	4	2	2	Strong (GenProg)	Moderate	High
Learning/neural repair	3	0	3	Moderate	Moderate	Medium
LLM-based debugging	6	3	3	n/a	Strong	High
Agentic / debugger-control	8	4	4	n/a	Strong	High
Multi-agent debugging	4	1	3	n/a	Moderate	Medium
Benchmarks/datasets	9	4	5	Strong	Strong	High

The map reveals the weakest areas: **learning/neural repair** (no full-text reads; relied on surveys and secondary citations), **root-cause-analysis evaluation methodology** (foundational but no modern RCA-correctness metric found), and the newest **debugger-control/multi-agent frontier** (several 2025–2026 papers accessed only via subagent relay, not full text).

2. Research Scope and Method

The review targets six questions: what has been done across debugging/APR/LLM-debugging/agents; which systems are most relevant to an agentic debugger; the static-vs-dynamic distinction; relevant datasets/benchmarks; what a first prototype should focus on; and what to read first. Method: iterative web_search + primary-source reading (arXiv, ACM DL, IEEE, OpenReview, official repos), followed by one targeted subagent investigation into the thinnest area (interactive debugger-control agents and multi-agent evidence), then an enrichment pass. Approximately 18 web searches were executed plus one subagent with its own search budget. English-language primary sources were prioritized; no Turkish sources were used.

3. Source Access and Verification Policy

Access levels: FULL_TEXT_READ (read the full text/HTML); SUBSTANTIAL_SECTIONS_READ (read multiple substantive sections/PDF extracts); ABSTRACT_ONLY; METADATA_ONLY; INACCESSIBLE. Because tool search returns snippets plus some full HTML/PDF extracts, most arXiv HTML papers where I read extended extracts are marked SUBSTANTIAL_SECTIONS_READ; papers seen only via abstract pages are ABSTRACT_ONLY. I

have not claimed full reading of any paper seen only in snippet form. Several 2026-dated arXiv preprints surfaced by the subagent are labeled as such and their author lists were not all verified.

4. Historical Development of the Field

The progression the project cares about can be traced as:

1. **Manual debugging** — hypothesis-driven inspection with breakpoints, stepping, and state inspection (the human baseline).
2. **Delta debugging / automated cause isolation** (Zeller, 1999–2002) — algorithmic isolation of failure-inducing input and *cause-effect chains* in program state.
3. **Spectrum-based fault localization** (Tarantula, 2005; Ochiai) — statistical ranking of suspicious statements from test coverage spectra.
4. **Automated program repair** (GenProg, 2009/2012) — generate-and-validate repair via genetic programming against a test suite; SemFix (semantic/constraint-based), PAR (template-based).
5. **Learning-based / neural repair** (SequenceR seq2seq; later CodeBERT/CodeT5-based) — treat repair as translation.
6. **LLM-based debugging/repair** (Self-Debugging 2023; LDB 2024) — prompting and execution-feedback iteration.
7. **Repository-level SE agents** (SWE-bench 2023; SWE-agent, OpenHands, AutoCodeRover, Agentless 2024) — issue-to-patch on real repos.
8. **Tool-using/agentic debugging** (RepairAgent 2024/2025) — autonomous tool selection loops.
9. **Interactive debugger-controlling agents** (ChatDBG 2024; debug-gym, EnIGMA 2025; FramePilot, Debug2Fix, SWE-Doctor 2026) — LLM actually drives a live debugger.

5. Core Concepts and Terminology

- **Debugging vs testing/verification/monitoring**: Testing detects the *presence* of failures; verification proves properties; monitoring/observability report runtime behavior; **debugging** is the diagnostic activity of locating and explaining the fault behind an observed failure, then fixing it. Static analysis reasons about code without executing it.
- **Fault localization**: producing a *ranked list* of suspicious code locations (statement/method/file). Output is a ranking, generally not a causal explanation.
- **Root-cause analysis**: identifying the actual causal origin of the failure — the variables/values/decisions along a cause-effect chain (Zeller). Requires data/control-flow and counterfactual reasoning.
- **Automated program repair (APR)**: generating candidate patches and validating them (usually against tests). **Plausible patch** = passes the test suite; **correct patch** = semantically matches developer intent. **Overfitting** = plausible-but-incorrect.

- **Suspiciousness score**: a formula (Tarantula, Ochiai, DStar, Jaccard, Op2, Barinel) over four counts (executed/not-executed × failing/passing).
- **Metrics**: Top-N accuracy, Mean Reciprocal Rank (MRR), EXAM score (fraction of code inspected before the fault), pass@k, resolved-rate (SWE-bench Fail-to-Pass).
- **Agentic loop**: reasoning-action-observation (ReAct-style) with tools, environment feedback, and iteration.

6. Traditional Debugging

Manual debugging is hypothesis-driven: reproduce the failure, form a hypothesis, set breakpoints, step through execution, inspect stack frames and variables, compare expected vs actual program state, refine the hypothesis, and fix. It is difficult because failures are often separated in time and space from their root causes, state spaces are large, and reproduction can be non-deterministic. The LDB paper explicitly motivates its design by noting that "when human developers debug programs, they typically set breakpoints and selectively examine runtime execution information" — a workflow underused in LLM code generation. This human loop is the template the intended system aims to automate. (arXiv)

7. Automated Debugging

Automated debugging historically automated *parts* of the loop rather than the whole. Key technique families:

- **Delta debugging** (Zeller/Hildebrandt): systematically minimizes failure-inducing input and isolates cause-effect chains by comparing passing vs failing runs and altering program state. This is the earliest rigorous *automated root-cause* line — it delivers causal explanation, not just ranking. Zeller's 2002 GCC case study isolated a chain from a `1.0` addition through the RTL representation to a cycle that crashed the compiler.
- **Fault localization** (Section 8).
- **Program slicing** (static/dynamic): data/control-dependency reduction of candidate code.
- **Algorithmic debugging**: interactive query over execution.

Automated debugging differs from fully autonomous repair: localization/isolation tools inform a human, whereas APR closes the loop by producing and validating a fix. Assumptions vary: SBFL needs a test suite with passing and failing cases; delta debugging needs a reproducible passing and failing run; slicing needs execution traces.

8. Fault Localization

Spectrum-based fault localization (SBFL) is the dominant classical family. It runs the test suite, records which program entities each test executes (the *spectrum*), and applies a suspiciousness formula. Representative formulas: **Tarantula** (Jones & Harrold 2005), **Ochiai**, **Jaccard**, **DStar**

(D*), **Op2**, **Barinel**, **Kulczynski**. All are functions of four counts: executed-failing (ef), not-executed-failing (nf), executed-passing (ep), not-executed-passing (np). Entities are ranked by suspiciousness for developer inspection. (arxiv)

Other families:

- **Statistical/predicate-based** (e.g., CBI-style predicate evaluation).
- **Mutation-based** (MUSE, Metallaxis): perturb the program and observe test-outcome changes.
- **Slicing-based**: dynamic slices from the failing run.
- **Dynamic invariant-based** (Daikon-style).
- **Learning-based** (MULTRIC combines multiple metrics; models learn to rank).
- **Neural/transformer-based** (e.g., LLMAO-type deep FL).
- **LLM-based / repository-level**: AutoCodeRover uses code search over program structure + optional SBFL; Agentless uses hierarchical LLM localization (files → classes/functions → edit locations). (Hugging Face)

Metrics: Top-1/Top-5/Top-10, MRR, EXAM. **Benchmarks**: Defects4J, BugsInPy, SIR, Siemens suite, ConDefects (leakage-aware).

Critical distinction for the project: SBFL and most FL techniques *rank* suspicious locations; they do NOT explain *why* the location is faulty. As the PageRank-SBFL literature notes, an ideal SBFL technique "should always rank the faulty entity with high suspiciousness score" — the deliverable is a rank, not a cause. Only delta-debugging and some invariant/causal approaches attempt explanation. Practical limitations: coincidentally-correct tests degrade accuracy; multiple faults confound ranking; SBFL depends heavily on test-suite quality; and Defects4J "data cleanness" studies show benchmark artifacts can distort FL evaluation. (Illinois)

9. Root-Cause Analysis

Root-cause analysis (RCA) differs from FL in kind, not degree. FL answers "where should I look?"; RCA answers "what causal chain produced this failure?" The **Zeller delta-debugging** line is the canonical automated RCA approach: by narrowing the state difference between passing and failing runs, it isolates the *variables and values* that cause the failure — an explicit cause-effect chain. This requires control flow, data flow, execution history, program state, and counterfactual reasoning ("if this variable had this value, does the failure disappear?"). (ResearchGate)

Among LLM systems, **ChatDBG** explicitly claims root-cause analysis: it lets the LLM drive the debugger to "perform root cause analysis for crashes or assertion failures" and answer "why is x null?" — leveraging runtime state plus real-world knowledge. However, evaluating RCA *correctness* is hard: to claim a model found the *true* root cause, one needs evidence that the identified cause is both necessary and sufficient for the failure (ideally demonstrated counterfactually via a fix that removes exactly that cause). Most LLM debugging papers evaluate

downstream *repair success* (tests pass), which is weaker evidence of true root-cause identification. This is a genuine research gap: no widely accepted RCA-correctness metric exists. arXiv

10. Automated Program Repair

GenProg (Le Goues, Nguyen, Forrest, Weimer; ICSE 2009, TSE 2012, DOI 10.1109/TSE.2011.104) is the seminal generate-and-validate APR system: genetic programming evolves program variants using existing test suites to encode required behavior and the defect, with structural differencing + delta debugging to minimize the patch. The TSE 2012 paper reports success on 16 programs totaling 1.25M lines of C code plus 120K lines of module code, spanning eight classes of defects, in 357 seconds on average. University of Michigan Ucl

Technique families:

- **Search-based / genetic** (GenProg lineage; PAR templates; TrpAutoRepair; SimFix).
- **Semantic/constraint-based / synthesis** (SemFix: symbolic execution + constraint solving + synthesis).
- **Template-based** (PAR; TBar; GAMMA via mask prediction).
- **Learning-based** (see Section 11).

Core concepts: the **fault space** (where to edit) and **patch space** (what edits); **plausible vs correct** patches; **patch overfitting** to the test suite (Smith et al., "Is the Cure Worse than the Disease? Overfitting in Automated Program Repair"); **test-suite adequacy**; **regression risk** (mitigated by regression testing); **developer acceptance** and **explainability**. Overfitting is the central validity threat: a weak test suite lets semantically wrong patches pass.

11. Learning-Based and Neural Program Repair

Neural APR reframes repair as sequence transduction. **SequenceR** (Chen et al., TSE 2018/2019) applied seq2seq with copy mechanisms to end-to-end repair. Subsequent work used pretrained code models (CodeBERT, CodeT5, PLBART) and code language models; an empirical study found the best code-LM fixed substantially more bugs than prior DL-based APR. RewardRepair, KNOD (domain-knowledge distilled tree decoder), DEAR, and Tare (2023 APR competition winner) are notable. The two systematic surveys — Zhang et al., "A Systematic Literature Review on Large Language Models for Automated Program Repair" (arXiv 2405.01466, later TOSEM), and Zhang et al., "A Survey of Learning-Based Automated Program Repair" (TOSEM 2023) — document the shift from seq2seq to LLM-based repair, with the LLM4APR SLR analyzing 189 papers across 2020–2025. Neural repair is mostly static (function-level, no runtime state) and inherits overfitting risks. (Note: this section relied on surveys and secondary citations, not full-text reads of the primary neural-repair papers — a coverage gap.)

12. LLM-Based Debugging

Classification of major LLM debugging approaches by context/dynamism/tools:

- **Self-Debugging** (Chen, Lin, Schärli, Zhou; Google Research and UC Berkeley; ICLR 2024; arXiv 2304.05128): teaches an LLM to debug its own generated code via few-shot prompting, using code explanation ("rubber duck") and execution results. Static-to-lightly-dynamic (uses execution results/unit tests when available); prompting-based; no debugger; can revise using feedback; function-level. Reported improvement of the baseline accuracy by up to 12% on TransCoder and MBPP (where unit tests are available); it can match or exceed baselines that sample >10× more candidates. (arXiv + 2)
- **LDB — Large Language Model Debugger** (Zhong, Wang, Shang; "Debug like a Human," ACL 2024 Findings; arXiv 2402.16906): segments the program into basic blocks, tracks intermediate variable values after each block during runtime execution, and verifies each block against the task description. This is genuinely **runtime-state-assisted** (though it instruments execution rather than driving an interactive debugger). Reported up to +9.8% over baseline across HumanEval, MBPP, TransCoder. Function/script level. (arXiv) (arXiv)
- **DebugBench** (Tian et al., ACL 2024 Findings; arXiv 2401.04621): an evaluation benchmark (below). Its finding that "incorporating runtime feedback has a clear impact on debugging performance which is not always helpful" is important — echoing debug-gym. (arxiv)
- **ChatDBG** (Section 16.1) is the clearest LLM system that controls a real debugger.

Caution demanded by the task: static LLM code repair (Self-Debugging, most SWE-bench agents) must NOT be labeled "dynamic debugging." LDB uses runtime *state* but not an interactive debugger; ChatDBG/debug-gym use an actual debugger.

13. Agentic and Tool-Using Debugging

Agentic systems add a reasoning-action-observation loop with tools (file read, code search, shell, test execution, patch application), memory, and iteration.

- **SWE-agent** (Section 16.2): repository navigation, file editing, test execution via an Agent-Computer Interface (ACI). No interactive debugger in the base system. Static-plus-test-feedback. (arXiv)
- **OpenHands** (Section 16.3): general platform; CodeAct agent executes bash/Python, browses web; sandboxed; multi-agent delegation supported. (arXiv)
- **AutoCodeRover** (Section 16.4): code search over AST structure + optional SBFL; static. (arXiv)
- **Agentless** (Section 16.5): deliberately non-agentic three-phase pipeline.
- **RepairAgent** (Bouzenia, Devanbu, Pradel; ICSE 2025; arXiv 2403.17134): the first *autonomous LLM agent* for APR. It dynamically updates its prompt, autonomously invokes tools (read file ranges, extract methods, search keywords, extract failing test code, compute

GZoltar SBFL scores), formulates hypotheses, generates and tests fixes in a loop. On Defects4J it produced plausible fixes for 186 bugs and **correct fixes for 164 bugs** (39 not fixed by prior tools), at ~270,000 tokens/bug (~$0.14/bug with GPT-3.5). It uses test feedback and SBFL but does NOT drive an interactive debugger (no breakpoints/stepping/variable inspection at runtime). arXiv + 4

True debugger control (the project's core interest):

- **ChatDBG**: LLM issues commands to LLDB/GDB/WinDBG (native) and PDB (Python) to navigate stacks and inspect state. arXiv
- **debug-gym** (Microsoft Research; arXiv 2503.21557): a text environment exposing `pdb` (breakpoints, step, continue, print, stack-frame eval), plus `eval`, `view`, `rewrite`, `listdir`, in a Docker sandbox. On SWE-bench-Lite, Claude 3.7 Sonnet improved from 37.2% (rewrite, no debugger) to 48.4% (with debugger) to 52.1% with the "debug(5)" variant (pdb made available only after the 5th rewrite attempt; ±1.6 SD over three runs). Crucially, pdb access *harmed* weaker models (GPT-4o, GPT-4o-mini, Llama-3.3-70B) when given from the start. Simon Willison arxiv
- **EnIGMA** (SWE-agent extension; ICML 2025; arXiv 2409.16165): adds interactive tools including a **GDB** interface (breakpoints, register/memory inspection, single-stepping) and identifies the "soliloquizing" failure mode (agent hallucinating tool output). Domain is CTF security, not repo bug-fixing.
- **FramePilot / ADI** (FSE 2026; arXiv 2604.24212): function-level PDB interaction via a "Frame Lifetime Trace"; resolves 63.8% of SWE-bench Verified at ~$1.28/task with Claude 3.7 Sonnet, and as a plug-in adds +6.2%–+18.5% to mini-SWE-agent and AutoCodeRover. Argues naive line-by-line debugger interaction is cost-inefficient for LLMs.
- **Debug2Fix** (arXiv 2602.18571): PDB + JDB via a debug subagent; improvement correlates with tool-call rate (GPT-5 ~16% gain).
- **SWE-Doctor** (arXiv 2607.00990): PDB + Delve (Go); 36.0% on SWE-bench Pro Go vs mini-SWE-agent 32.0%.
- **Debug Adapter Protocol (DAP) + LLM**: only engineering tooling (CLIs, MCP servers, VS Code extensions), no primary academic paper; debug-gym explicitly lists DAP generalization as future work.

14. Multi-Agent Debugging

Multi-agent designs assign roles (planner, localizer, patch generator, tester, critic/reviewer, repo navigator). Evidence on whether they help is genuinely mixed:

- **FixAgent / UniDebugger** (arXiv 2404.17153): hierarchical multi-agent (up to seven specialized agents), fixes 79/80 QuixBugs, 1.9×–2.2× more Codeflaws defects than best baseline, 368/600 ConDefects. But the paper itself concedes multi-agent feedback "does not

significantly enhance the debugging capability of LLMs compared to independent sampling."

- **MASAI** (Arora et al., Microsoft Research India; arXiv 2406.11638): five modular sub-agents; 28.33% on SWE-bench Lite at <$2/issue; 75% file-level localization.
- **Agentless** (Xia et al., FSE 2025; arXiv 2407.01489): NON-agentic pipeline; 27.33%→32.00% on SWE-bench Lite at $0.34–0.70/issue — matches/beats multi-agent systems at 3–9× lower cost. The direct counter-evidence. Hugging Face + 2
- The SWE-agent team reportedly abandoned multi-agent flows for SWE tasks (multi-agent helped EnIGMA's CTF tasks because they decompose more cleanly).

Verdict: multi-agent setups can help on decomposable tasks and localization, but the accuracy premium over well-designed single-agent/agentless approaches is small-to-none and carries clear cost/latency penalties (agentic methods cost roughly 3–9× more per instance than lightweight single-pass methods). Multi-agent superiority must not be assumed.

15. Static vs Dynamic Debugging Taxonomy

A precise ladder (each level adds information and risk/cost):

1. **Static code analysis** — code only; no execution. (classic linters, static slicing)
2. **LLM analysis of static code** — LLM reads code. (base function-level repair)
3. **Error-message-assisted static analysis** — code + stack trace/error text. (many SWE-bench agents' first step)
4. **Test-feedback-based iterative repair** — code + test pass/fail output, iterated. (Self-Debugging, Agentless, SWE-agent, AutoCodeRover, RepairAgent)
5. **Execution-trace-assisted analysis** — code + coverage spectra/traces. (SBFL; AutoCodeRover with SBFL)
6. **Runtime-state-assisted debugging** — code + intermediate variable values. (LDB)
7. **Interactive debugger-assisted debugging** — LLM issues debugger commands. (ChatDBG, debug-gym, EnIGMA, FramePilot, Debug2Fix, SWE-Doctor)
8. **Autonomous debugger control** — agent independently drives the full debug loop and closes it with a validated patch. (aspirational; partially realized by ChatDBG + debug-gym baselines)

New information at each level: level 3 adds the failure symptom; level 4 adds pass/fail oracle signal; level 5 adds which code ran; level 6 adds concrete values; level 7 adds *on-demand, interactive* querying of arbitrary state; level 8 adds autonomous hypothesis-testing. New risks: added tokens/cost/latency, tool-use errors, "soliloquizing" (hallucinated tool output), and — per debug-gym — weaker models being *harmed* by extra capability.

System classification: SWE-agent/OpenHands/AutoCodeRover/Agentless/RepairAgent = levels 3–5; LDB = level 6; ChatDBG/debug-gym/EnIGMA/FramePilot/Debug2Fix/SWE-Doctor = level 7.

16. System Case Studies

16.1 ChatDBG

- **Paper**: "ChatDBG: Augmenting Debugging with Large Language Models" (arXiv 2403.16354; Proc. ACM SE, DOI 10.1145/3729355). Authors: Kyla Levin, Nicolas van Kempen, Emery D. Berger, Stephen N. Freund (UMass Amherst / Williams College). Repo: github.com/plasma-umass/ChatDBG. Access: SUBSTANTIAL_SECTIONS_READ.
- **Goal/architecture**: AI-powered debugging assistant that grants the LLM autonomy to "take the wheel" and drive standard debuggers. (arXiv)
- **Debuggers**: LLDB, GDB, WinDBG (native C/C++), PDB (Python). **This is true debugger control.** (arXiv)
- **Tools**: debugger commands to navigate stacks, inspect program state; answers "why is x null?"-style queries. (arXiv)
- **Static/dynamic**: Level 7 (interactive debugger-assisted). Model can request more runtime information by issuing commands.
- **Evaluation**: C/C++ programs with known bugs; Python scripts and Jupyter notebooks. Reported to significantly assist in identifying root causes and correct fixes (qualitative + case studies). Model used: GPT-4 family. (arXiv + 2)
- **Relevance**: Highest — it is the closest existing artifact to the intended system, and covers PDB (the recommended first target).

16.2 SWE-Agent

- **Paper**: "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering" (arXiv 2405.15793; NeurIPS 2024). Authors: John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, Ofir Press (Princeton). Access: SUBSTANTIAL_SECTIONS_READ.
- **Goal/architecture**: Agent-Computer Interface (ACI) letting an LM navigate repos, create/edit files, run tests. Base LM: GPT-4 Turbo. (arXiv)
- **Tools**: file viewer/editor, directory search, shell/test execution. No interactive debugger in base.
- **Static/dynamic**: Levels 3–4 (error text + test feedback).
- **Evaluation**: SWE-bench; ~12.5% resolved (GPT-4 Turbo), vs 3.8% best prior RAG system. (Medium)
- **Relevance**: High as a comparison point for repo navigation + ACI design; not a dynamic debugger.

16.3 OpenHands (formerly OpenDevin)

- **Paper**: "OpenHands: An Open Platform for AI Software Developers as Generalist Agents" (arXiv 2407.16741; ICLR 2025). Lead: Xingyao Wang et al. (24 authors), Graham Neubig (UIUC/CMU + community). MIT license. Access: SUBSTANTIAL_SECTIONS_READ.
- **Goal/architecture**: platform for generalist agents; event-stream architecture; sandboxed runtime; CodeAct agent executes bash/Python; multi-agent delegation via AgentDelegateAction; 15+ evaluation benchmarks. (arXiv) (Illinois Experts)
- **Tools**: bash, Python execution, file editing (adapted from SWE-agent/Aider), web browsing, multimodal parsing. (arXiv)
- **Static/dynamic**: Levels 3–4; can execute code but not a standard interactive debugger loop by default.
- **Relevance**: High as an extensible platform on which a debugger tool could be built; strong open-source engineering base.

16.4 AutoCodeRover

- **Paper**: "AutoCodeRover: Autonomous Program Improvement" (arXiv 2404.05427; ISSTA 2024, DOI 10.1145/3650212.3680384). Authors: Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, Abhik Roychoudhury (NUS). Access: SUBSTANTIAL_SECTIONS_READ.
- **Goal/architecture**: issue → patch via code search over program structure (AST classes/methods) + iterative context retrieval; optional spectrum-based fault localization when tests exist. (ACM Digital Library)
- **Results**: 19% (v1) then 22–23% on SWE-bench Lite; ~16% on full SWE-bench; ~$0.43/issue; ~57–67 issues resolved in minutes each. (ACM Digital Library + 2)
- **Static/dynamic**: Levels 3–5 (structure + SBFL). Not a runtime debugger.
- **Relevance**: High for its structure-aware localization and SBFL integration — directly reusable ideas.

16.5 Agentless

- **Paper**: "Agentless: Demystifying LLM-based Software Engineering Agents" (arXiv 2407.01489; FSE 2025, DOI 10.1145/3715754). Authors: Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, Lingming Zhang (UIUC). Access: SUBSTANTIAL_SECTIONS_READ.
- **Goal/architecture**: deliberately NON-agentic three-phase pipeline: hierarchical localization → repair → patch validation; LLM never autonomously chooses actions or uses complex tools. (ACM Digital Library)
- **Results**: v1 GPT-4o 27.33% ($0.34/issue); updated 32.00% (96/300, $0.70); Claude 3.5 Sonnet ~40.7% Lite / >50% Verified. Adopted by OpenAI and DeepSeek for SWE-bench

evaluation. Also constructed SWE-bench Lite-S excluding problematic issues. Hugging Face + 3

- **Relevance**: High as the baseline that disciplines any agentic ambition — proves simple can win on cost/accuracy.

16.6 Other Important Systems Found During Research

- **RepairAgent** (ICSE 2025; arXiv 2403.17134): first autonomous LLM APR agent; Defects4J 164 correct fixes; tools + SBFL, no interactive debugger. Access: SUBSTANTIAL_SECTIONS_READ.
- **LDB** (ACL 2024; arXiv 2402.16906): runtime-state block-level verification; +9.8%. Access: SUBSTANTIAL_SECTIONS_READ.
- **Self-Debugging** (ICLR 2024; arXiv 2304.05128): prompting-based self-repair; up to +12% on TransCoder/MBPP. Access: SUBSTANTIAL_SECTIONS_READ.
- **debug-gym** (arXiv 2503.21557, Microsoft Research): pdb-based interactive debugging environment; Claude 3.7 52.1% SWE-bench-Lite. Access: METADATA via subagent (findings relayed).
- **EnIGMA** (ICML 2025; arXiv 2409.16165): SWE-agent + GDB for CTF. Access: METADATA via subagent.
- **FramePilot/ADI** (FSE 2026; arXiv 2604.24212): function-level PDB; 63.8% SWE-bench Verified. Access: METADATA via subagent.
- **Debug2Fix** (arXiv 2602.18571) and **SWE-Doctor** (arXiv 2607.00990): PDB+JDB and PDB+Delve subagent debugging. Access: METADATA via subagent; author lists unverified.
- **FixAgent/UniDebugger** (arXiv 2404.17153) and **MASAI** (arXiv 2406.11638): multi-agent. Access: METADATA via subagent.

17. Dataset and Benchmark Review

- **SWE-bench** (Jimenez et al., ICLR 2024; arXiv 2310.06770): 2,294 real GitHub issue→PR tasks across 12 Python repos; Fail-to-Pass + Pass-to-Pass tests; Docker per instance. On release, the best-performing model, Claude 2 (with a BM25 retriever), resolved just 1.96% of issues. Includes issue text, tests, bug-fix commits. Leakage concerns documented (below). arxiv + 2
- **SWE-bench Lite**: 300-instance curated subset; de facto agent benchmark. arxiv
- **SWE-bench Verified**: 500 human-validated solvable tasks (OpenAI collaboration). Now heavily critiqued: per Aleithan et al., SWE-Bench+ (arXiv 2410.06992), 32.67% of successful patches involve "cheating" via solution leakage and 31.08% of passed patches are suspicious due to weak test cases (based on manual screening of SWE-Agent+GPT-4 patches); and 94% of instances and their pull requests were created prior to the training cut-off dates of the LLMs. SOTA models identify buggy file paths from issue text alone at up to 76% (vs 53% on

external repos), indicating memorization (The SWE-Bench Illusion, arXiv 2506.12286). OpenAI itself recommended discontinuing SWE-bench Verified reporting after an audit found 59.4% of o3 failures were caused by test flaws. GitHub + 2

- **Defects4J** (Just et al., ISSTA 2014, DOI 10.1145/2610384.2628055): originally 357 real Java bugs / 5 projects; v2.0 has 835 bugs / 17 projects; each with a fault-triggering test. The workhorse for FL and APR. Data-cleanness studies flag artifacts. Typeset arXiv
- **BugsInPy** (Widyasari et al., ESEC/FSE 2020; arXiv 2401.15481): 493 real bugs from 17 Python projects with tests; Defects4J-inspired. Most relevant Python real-bug benchmark for the project. arXiv
- **QuixBugs** (Lin et al., SPLASH Companion 2017, DOI 10.1145/3135932.3135941): 40 small programs in Python and Java, one-line defects, with tests. Good for cross-language single-line repair; synthetic, small. ACM Digital Library
- **DebugBench** (Tian et al., ACL 2024 Findings; arXiv 2401.04621): 4,253 instances, C++/Java/Python, 4 major/18 minor bug types, GPT-4-implanted bugs from LeetCode; leakage-aware. Includes runtime-feedback experiments. arxiv
- **HumanEvalFix** (part of OctoPack, Muennighoff et al.): function-level bug-fix benchmark across languages with tests; useful for quick evaluation of static repair (metadata-level; not deeply read).
- **GitBug-Java**: recent executable Java bug benchmark (referenced by Debug2Fix); metadata-level.
- **ConDefects**: leakage-aware FL/APR dataset (referenced).

18. Approach Comparison Matrix

Dimension	Manual	Classical automated (delta/SBFL)	APR (GenProg)	LLM-assisted (Self-Debug)	RAG-supported	Tool-using agent (SWE-agent/RepairAgent)	Mu ag
Required inputs	repro + human	tests/traces	test suite	code + tests	code + retrieval corpus	repo + tests	rep tes
Static/dynamic	dynamic	dynamic	dynamic (validation)	mostly static	static	static+test feedback	sta fee
Runtime-state access	Yes (human)	Partial (state diffs)	No (only test outcome)	No	No	No	No

Dimension	Manual	Classical automated (delta/SBFL)	APR (GenProg)	LLM-assisted (Self-Debug)	RAG-supported	Tool-using agent (SWE-agent/RepairAgent)	Mu ag
Repo awareness	Yes	Limited	Limited	No	Partial	Yes	Yes
Request more evidence	Yes	No	No	Limited	No	Yes	Yes
Localization	Human	Yes (ranking)	via FL	Weak	Weak	Yes	Yes
Root-cause	Yes	Yes (delta)	No	Weak	No	Partial	Par
Patch generation	Human	No	Yes	Yes	Yes	Yes	Yes
Test validation	Manual	n/a	Yes	Yes	Optional	Yes	Yes
Iterative repair	Yes	No	Yes (GP)	Yes	Limited	Yes	Yes
Explainability	High	High (causal)	Low	Medium	Low	Medium	Me
Human involvement	Full	Low	Low	Low	Low	Low	Lo
Cost/latency	High (human)	Low	High	Low	Low	Medium-High	Hig
Reproducibility	Low	High	Medium	Medium	Medium	Medium	Lo Me
Small-model suitability	n/a	High	Medium	Medium	Medium	Low-Medium	Lo

19. System Capability Matrix

System	Static code	Error text	Tests	Exec trace	Runtime vars	Stack frames	Interactive debugger	Repo nav	Code search
ChatDBG	Yes	Yes	Partial	Partial	Yes	Yes	**Yes**	No	No
SWE-agent	Yes	Yes	Yes	No	No	No	No	Yes	Yes
OpenHands	Yes	Yes	Yes	No	Partial	No	No	Yes	Yes
AutoCodeRover	Yes	Yes	Yes	Yes(SBFL)	No	No	No	Yes	Yes
Agentless	Yes	Yes	Yes	No	No	No	No	Partial	Yes
RepairAgent	Yes	Yes	Yes	Yes(SBFL)	No	No	No	Yes	Yes
LDB	Yes	Yes	Yes	Yes	**Yes**	No	No	No	No
Self-Debugging	Yes	Yes	Partial	No	No	No	No	No	No
debug-gym	Yes	Yes	Yes	Partial	Yes	Yes	**Yes**	Yes	Partial
EnIGMA	Yes	Yes	Yes	Partial	Yes	Yes	**Yes (GDB)**	Yes	Yes
FramePilot/ADI	Yes	Yes	Yes	Yes	Yes	Yes	**Yes (PDB)**	Yes	Yes
MASAI	Yes	Yes	Yes	No	No	No	No	Yes	Yes

20. Dataset Suitability Matrix

Dataset	Languages	Task type	Tests	Issues	Bug-fix commits	Traces/errors	SFT
SWE-bench	Python	issue→patch	Yes	Yes	Yes	Partial	Partial
SWE-bench Lite	Python	issue→patch	Yes	Yes	Yes	Partial	No
SWE-bench Verified	Python	issue→patch	Yes	Yes	Yes	Partial	No
Defects4J	Java	bug+test	Yes	No	Yes	Partial	Partial

Dataset	Languages	Task type	Tests	Issues	Bug-fix commits	Traces/errors	SFT
BugsInPy	Python	bug+test	Yes	Partial	Yes	Partial	Partial
QuixBugs	Python/Java	1-line repair	Yes	No	Yes	No	No
DebugBench	C++/Java/Python	bug fix	Partial	No	No (implanted)	Partial	Partial
HumanEvalFix	multi	func fix	Yes	No	No	No	Partial
GitBug-Java	Java	bug+test	Yes	Partial	Yes	Partial	Partial

21. Evaluation Methods and Metrics

- **Fault localization**: Top-N accuracy, MRR, EXAM, wasted effort.
- **Repair**: plausible-fix count (tests pass), correct-fix count (semantic match to developer patch), pass@k, resolved-rate (SWE-bench Fail-to-Pass + Pass-to-Pass).
- **Cost/efficiency**: tokens/bug, USD/issue, wall-clock.
- **Contamination controls**: temporal splits (post-cutoff issues), decontaminated benchmarks (SWE-rebench, SWE-bench Pro), file-path-from-issue-only diagnostics.
- **Key caveat**: resolved-rate conflates true fixes with leakage/weak-test artifacts; correct-fix (manual verification) is the stronger metric, used by RepairAgent and FixAgent's ConDefects evaluation.

22. Major Findings

1. Runtime/debugger information is additive but conditional — helps strong models, can harm weak ones (debug-gym).
2. Fault localization ≠ root-cause analysis; only the delta-debugging lineage and (aspirationally) debugger-driven LLM agents attempt genuine causal explanation.
3. Passing tests ≠ correct patch; overfitting and SWE-bench leakage/weak-tests are pervasive and quantified.
4. Simple pipelines (Agentless) rival complex/multi-agent systems at far lower cost.
5. The interactive-debugger-control lineage is small but growing fast (ChatDBG → debug-gym/EnIGMA → FramePilot/Debug2Fix/SWE-Doctor).
6. Python/PDB is the most common and best-supported first target for debugger control.

7. DAP+LLM integration is an open engineering/research gap with no primary academic paper yet.

23. Contradictions, Uncertainties, and Evidence Strength

- **Multi-agent value**: FixAgent claims large gains; Agentless and the SWE-agent team's experience contradict a general accuracy premium. Evidence: Moderate, mixed.
- **Debugger benefit**: debug-gym (helps strong models) vs DebugBench ("runtime feedback not always helpful"). Evidence: Moderate; effect is model- and integration-dependent.
- **SWE-bench validity**: multiple independent studies converge on contamination/leakage. Evidence: Strong.
- **RCA correctness in LLMs**: ChatDBG claims root-cause analysis, but no rigorous RCA-correctness metric exists. Evidence: Weak/Inconclusive.
- **MASAI resolved-rate**: 28.33% (paper) vs 27.33%/32.6% (leaderboard survey) — version discrepancy unresolved.

24. Research Gaps

Well-established gaps: FL-to-RCA gap; plausible-vs-correct patch gap and overfitting; limited runtime-state use in LLM repair; weak/absent debugger integration in mainstream agents; benchmark leakage/contamination/weak tests; cost and latency of agentic loops; hallucinated fixes; context-window/repo-scale navigation limits.

Inferred opportunities: unsupported causal explanations (RCA evaluation methodology); DAP-based debugger portability across languages; small/medium open-model suitability for debugger-driven loops; reproducible, contamination-resistant dynamic-debugging benchmarks; a principled controller that decides *when* to invoke the debugger (debug-gym's delayed-pdb result suggests timing matters).

25. Implications for the Agentic Debugging Project

- **Most relevant prior work**: ChatDBG (direct precedent, PDB control), debug-gym (environment + pdb tool design + the delayed-debugger insight), LDB (runtime-state extraction), RepairAgent (autonomous tool loop + SBFL), AutoCodeRover (structure-aware localization), Agentless (disciplined baseline).
- **Reusable components**: PDB wrapping (ChatDBG/debug-gym), SBFL scoring (GZoltar/Ochiai; RepairAgent), AST/code search (AutoCodeRover), sandboxed runtime + event stream (OpenHands), ACI design (SWE-agent), hierarchical localization + patch validation (Agentless).
- **Debugger interaction provides information unavailable to static agents** — this is supported (LDB runtime state; debug-gym gains for strong models) but conditional; the controller must decide when to use it.

- **First debugger: Python/PDB** — best evidence (ChatDBG, debug-gym, FramePilot, Debug2Fix all use it), simplest instrumentation, richest benchmarks (BugsInPy, Defects4J-analog workflows, SWE-bench-Lite).
- **Division of responsibility**: deterministic tools for spectrum/trace capture, breakpoint execution, and test running; the LLM for hypothesis formation, state interpretation, and patch drafting; the agent controller for tool orchestration and stopping; a **verifier** (regression + reproduction tests, ideally differential/augmented tests to fight overfitting) is mandatory.
- **Fine-tuning**: NOT justified initially — prompting/tool-use with strong models suffices per Agentless/debug-gym; fine-tuning is a later experiment, especially if targeting small open models.
- **RAG**: justified for repo-scale context retrieval (code search), less so for the debugging loop itself.
- **Multi-agent**: NOT justified initially — evidence favors a single well-instrumented agent; revisit only if decomposition demonstrably helps.
- **MVP scope**: single-language (Python), single-agent, PDB-driven loop on reproducible failing tests (BugsInPy / SWE-bench-Lite subset), with deterministic FL + verifier and a controller that gates debugger use.
- **Out of scope initially**: multi-language debugger adapters (DAP), multi-agent review, fine-tuning/DPO/RLHF, non-reproducible/production observability.

26. Recommended Next Research Steps

1. Reproduce ChatDBG and debug-gym on a small Python bug set (BugsInPy subset) to measure the runtime-info delta and the "harm-to-weak-models" effect on your chosen models.
2. Build a deterministic PDB tool layer (breakpoints, step, frame/variable inspection, expression eval) + SBFL scorer as the agent's action space.
3. Implement a verifier: reproduction test + regression tests + (stretch) differential/augmented tests to detect overfitting.
4. Evaluate on a contamination-controlled split (post-cutoff issues; SWE-rebench-style) to avoid leakage-inflated numbers.
5. Define an RCA-correctness protocol (counterfactual: does removing the identified cause fix the failure without breaking others?).
6. Only after a single-agent baseline is solid, ablate: delayed-vs-immediate debugger use; single vs multi-agent; small vs large model.

Thresholds that change the plan: if debugger access does not beat a strong Agentless-style baseline on correct-fix rate at acceptable cost, deprioritize interactive debugging; if small open models cannot use debugger tools productively (per debug-gym), defer local-model deployment.

27. Priority Reading List

TIER 1 — Must read completely

- ChatDBG (arXiv 2403.16354) — read full: architecture, debugger command interface, evaluation. Direct precedent.
- debug-gym (arXiv 2503.21557) — read full: tool design, delayed-pdb result, SWE-bench-Lite tables. Environment blueprint.
- Agentless (arXiv 2407.01489) — read full: localization/repair/validation phases; the baseline to beat.
- SWE-bench (arXiv 2310.06770) — read full: task construction, Fail-to-Pass metric; evaluation foundation.

TIER 2 — Read core sections

- LDB (arXiv 2402.16906) — runtime-state extraction method (Sections 2–3).
- RepairAgent (arXiv 2403.17134) — tool set + loop (Sections III–V).
- AutoCodeRover (arXiv 2404.05427) — code search + SBFL (Sections 2–4).
- SWE-agent (arXiv 2405.15793) — ACI design (Sections 2–4).
- Zeller, "Isolating Cause-Effect Chains" (FSE 2002) — RCA foundations.
- GenProg (TSE 2012) — APR foundations (Sections 1–4).
- LLM4APR SLR (arXiv 2405.01466) — taxonomy chapter.

TIER 3 — Useful supporting

- OpenHands (arXiv 2407.16741); DebugBench (arXiv 2401.04621); EnIGMA (arXiv 2409.16165); FramePilot (arXiv 2604.24212); Self-Debugging (arXiv 2304.05128); SBFL survey (arXiv 1607.04347).

TIER 4 — Background/optional

- QuixBugs; BugsInPy; Defects4J; SWE-bench contamination studies; FixAgent; MASAI; Debug2Fix; SWE-Doctor.

28. Full Source Inventory (Source Ledger)

ID	Title	Authors	Year	Venue	Type	Peer-reviewed	Access
S1	ChatDBG: Augmenting Debugging with LLMs	Levin, van Kempen, Berger, Freund	2024	Proc. ACM SE	paper	Yes	SUBSTAN

ID	Title	Authors	Year	Venue	Type	Peer-reviewed	Access
S2	SWE-agent	Yang et al.	2024	NeurIPS	paper	Yes	SUBSTAN
S3	AutoCodeRover	Zhang, Ruan, Fan, Roychoudhury	2024	ISSTA	paper	Yes	SUBSTAN
S4	Agentless	Xia, Deng, Dunn, Zhang	2024/25	FSE 2025	paper	Yes	SUBSTAN
S5	OpenHands	Wang et al.	2024/25	ICLR 2025	paper	Yes	SUBSTAN
S6	SWE-bench	Jimenez et al.	2023/24	ICLR 2024	paper	Yes	SUBSTAN
S7	SBFL survey	de Souza et al.	2016	arXiv	survey	No	SUBSTAN
S8	GenProg	Le Goues, Nguyen, Forrest, Weimer	2012	IEEE TSE	paper	Yes	SUBSTAN
S9	LDB	Zhong, Wang, Shang	2024	ACL Findings	paper	Yes	SUBSTAN
S10	Self-Debugging	Chen, Lin, Schärli, Zhou	2023/24	ICLR 2024	paper	Yes	SUBSTAN
S11	RepairAgent	Bouzenia, Devanbu, Pradel	2024/25	ICSE 2025	paper	Yes	SUBSTAN
S12	Defects4J	Just, Jalali, Ernst	2014	ISSTA	paper	Yes	SUBSTAN

ID	Title	Authors	Year	Venue	Type	Peer-reviewed	Access
S13	BugsInPy	Widyasari et al.	2020	ESEC/FSE	paper	Yes	SUBSTAN
S14	QuixBugs	Lin, Koppel, Chen, Solar-Lezama	2017	SPLASH Comp.	paper	Yes	ABSTRAC
S15	DebugBench	Tian et al.	2024	ACL Findings	paper	Yes	SUBSTAN
S16	Delta Debugging (cause-effect)	Zeller	2002	FSE	paper	Yes	SUBSTAN
S17	LLM4APR SLR	Zhang et al.	2024	arXiv/TOSEM	survey	Yes(later)	SUBSTAN
S18	SWE-Bench+ / contamination	Aleithan et al.; Liang et al.; OpenAI	2024-25	arXiv/blog	analysis	Mixed	SUBSTAN
S19	debug-gym	Yuan, Côté et al.	2025	arXiv (MSR)	tech report	No	METADAT
S20	EnIGMA	Abramovich et al.	2024/25	ICML 2025	paper	Yes	METADAT
S21	FramePilot/ADI	Xiang et al.	2026	FSE 2026	paper	Yes	METADAT
S22	FixAgent/UniDebugger	(unverified)	2024	arXiv	paper	No	METADAT
S23	MASAI	Arora et al.	2024	arXiv	paper	No	METADAT
S24	Debug2Fix	(unverified)	2026	arXiv	preprint	No	METADAT
S25	SWE-Doctor	(unverified)	2026	arXiv	preprint	No	METADAT

29. Inaccessible or Partially Unread Sources

- **debug-gym, EnIGMA, FramePilot, Debug2Fix, SWE-Doctor, FixAgent, MASAI** (S19–S25): findings relayed by subagent from primary sources; I did not personally read full texts. Abstracts available on arXiv. Relevant because they represent the debugger-control and multi-agent frontier. Author lists for Debug2Fix/SWE-Doctor/FixAgent not verified.
- **QuixBugs** (S14): abstract only.
- **HumanEvalFix/OctoPack, GitBug-Java, ConDefects**: metadata only.
- **SequenceR, SemFix, PAR, MULTRIC, TBar**: known via secondary citations; not independently read.

30. PDF Download Manifest

Priority	Suggested filename	Title	Authors	Year	Ide
1	2024_chatdbg_ai_powered_debugging_assistant.pdf	ChatDBG	Levin et al.	2024	arX
1	2025_debug_gym_interactive_debugging_env.pdf	debug-gym	Yuan, Côté et al.	2025	arX
1	2024_agentless_demystifying_se_agents.pdf	Agentless	Xia et al.	2024	arX
1	2023_swe_bench_resolve_github_issues.pdf	SWE-bench	Jimenez et al.	2023	arX
2	2024_ldb_llm_debugger_runtime.pdf	LDB	Zhong et al.	2024	arX
2	2024_repairagent_autonomous_apr.pdf	RepairAgent	Bouzenia et al.	2024	arX
2	2024_autocoderover_program_improvement.pdf	AutoCodeRover	Zhang et al.	2024	arX
2	2024_swe_agent_aci.pdf	SWE-agent	Yang et al.	2024	arX
2	2002_zeller_isolating_cause_effect_chains.pdf	Isolating Cause-Effect Chains	Zeller	2002	DOI 10.1
2	2012_genprog_generic_automatic_repair.pdf	GenProg	Le Goues et al.	2012	DOI 10.1
3	2024_llm4apr_systematic_review.pdf	LLM4APR SLR	Zhang et al.	2024	arX

Priority	Suggested filename	Title	Authors	Year	Ide
3	2024_openhands_platform.pdf	OpenHands	Wang et al.	2024	arX
3	2024_debugbench_llm_debugging.pdf	DebugBench	Tian et al.	2024	arX
3	2025_enigma_interactive_tools_security.pdf	EnIGMA	Abramovich et al.	2024	arX
3	2020_bugsinpy_python_bugs.pdf	BugsInPy	Widyasari et al.	2020	arX
3	2014_defects4j_database.pdf	Defects4J	Just et al.	2014	DOI 10.1

31. Claim-Evidence Matrix (Evidence Strength)

Claim ID	Claim	Supporting	Contradicting	Strength	Confidence	Not
C1	ChatDBG lets an LLM control real debuggers (LLDB/GDB/WinDBG/PDB)	S1	—	Strong	High	auth own
C2	Fault localization ranks, does not explain root cause	S7, S16	—	Strong	High	defi
C3	Delta debugging isolates causal cause-effect chains	S16	—	Strong	High	Zell
C4	GenProg is seminal generate-and-validate APR	S8	—	Strong	High	TSE
C5	Passing tests ≠ correct patch (overfitting)	S8, S17, S18	—	Strong	High	wid repl
C6	SWE-bench Verified has leakage/weak-test issues (32.67%/31.08%)	S18	—	Strong	High	incl. Ope
C7	Runtime/debugger info helps strong but can harm weak models	S19 (subagent)	S15(partly)	Moderate	Medium	deb

Claim ID	Claim	Supporting	Contradicting	Strength	Confidence	Not
C8	Simple (Agentless) rivals multi-agent at lower cost	S4	S22	Moderate-Strong	Medium-High	mix cons
C9	RepairAgent: 164 correct Defects4J fixes	S11	—	Strong	High	auth repo
C10	LLM RCA-correctness lacks a rigorous metric	(absence)	—	Inconclusive	Medium	gap
C11	Python/PDB is best-evidenced first target	S1,S9,S19,S21,S24	—	Moderate	Medium-High	conv usag

32. Research Self-Audit

- Total sources discovered: ~50+ (including secondary). Unique primary works after dedup: 25 (S1–S25).
- Full-text read: 0 claimed as complete; SUBSTANTIAL_SECTIONS_READ: 17 (S1–S13, S15–S18); ABSTRACT_ONLY: 1 (S14); METADATA_ONLY: 7 (S19–S25, via subagent relay); INACCESSIBLE: 0.
- Peer-reviewed: ~16; preprints/non-peer-reviewed at time of writing: ~9.
- Foundational coverage (Zeller, GenProg, Defects4J, Tarantula/SBFL): adequate. Recent coverage (2024–2026 agents/debugger-control): strong.
- Every material claim is cited to a source ID. Inaccessible/metadata-only sources are labeled. No paper was summarized beyond accessed sections; subagent-relayed items are explicitly flagged. Duplicate arXiv/venue versions merged (e.g., S1 v1/v3). System capability claims cross-checked against papers + repos where possible; contradictory evidence (multi-agent, debugger benefit) included. Direct PDF links marked "verify" or DIRECT_PDF_NOT_VERIFIED — not asserted as confirmed.
- Final search date: 18 July 2026.

Unresolved Items

A. Papers I could not access (full text): debug-gym, EnIGMA, FramePilot, Debug2Fix, SWE-Doctor, FixAgent, MASAI (abstracts/subagent only); QuixBugs (abstract only).

B. Claims I could not verify: exact author lists for Debug2Fix/SWE-Doctor/FixAgent; MASAI 28.33% vs 27.33%/32.6% discrepancy; some 2026-preprint figures; whether SWE-bench leakage figures (Aleithan et al.) apply identically to Verified vs full SWE-bench.

C. Questions requiring experiments: does debugger interaction beat a strong Agentless baseline on *correct-fix* rate at acceptable cost; can small open models use debugger tools productively; what RCA-correctness metric is valid; optimal timing of debugger invocation.

Verdict: LITERATURE_COVERAGE_ADEQUATE_WITH_GAPS — foundational and recent coverage is strong for the static/repository lineage and good for the emerging debugger-control lineage, but several frontier debugger-control and multi-agent papers were accessed only via subagent relay/abstracts, and no rigorous RCA-correctness literature was found, leaving genuine gaps that require full-text reading and experiments before design decisions are finalized.